

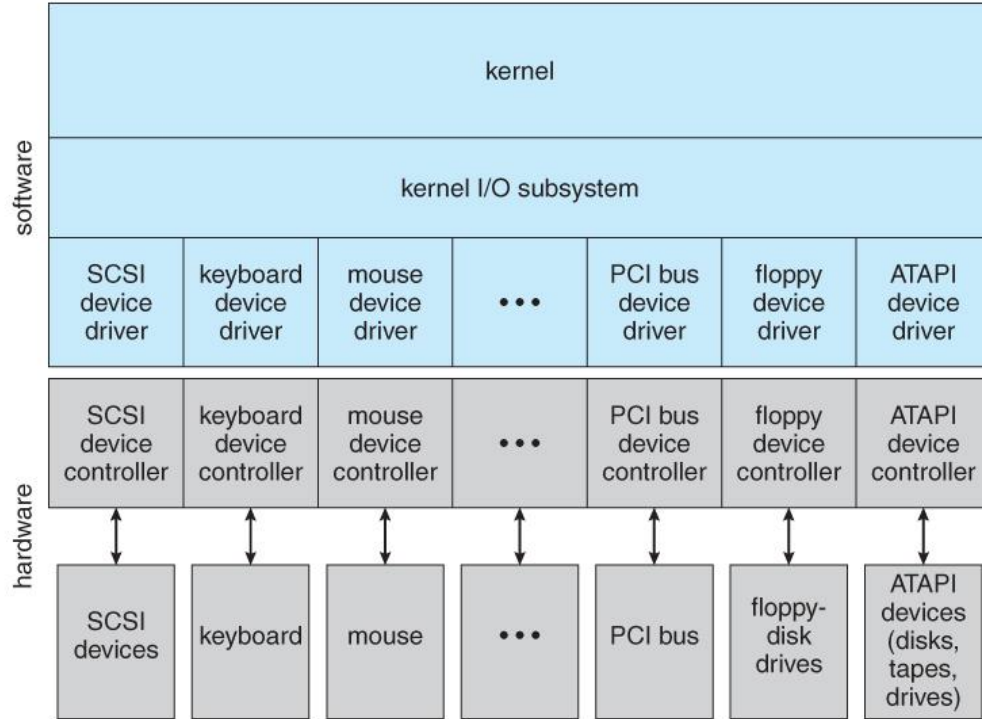
I/O Systems-II

Dr. Jit Mukherjee



Application I/O Interface

- I/O devices to be treated in a standard uniform way.
 - Abstract to find a general kind
 - Access -> Standardized -> Interface
 - Differences encapsulated -> kernel
- Device Driver Layer - hide the differences among device controller and I/O subsystem of kernel
 - Hide hardware differences from application
 - Easier to attach new peripherals
- Character-stream - transfer bytes
 - Block -> block by block
- Sequential - transfer in fixed order
 - Random-access - instruct the device to seek any available data
- Synchronous-Transfer predictable response time.
 - Asynchronous - irregular response time
- Sharable - can be used concurrently by several process. Dedicated -> can not
- Devices speed range from few byte to GB/sec
- Read-write, read only, write only
- Escape / Back Door -> passes arbitrary command from an application to a device driver



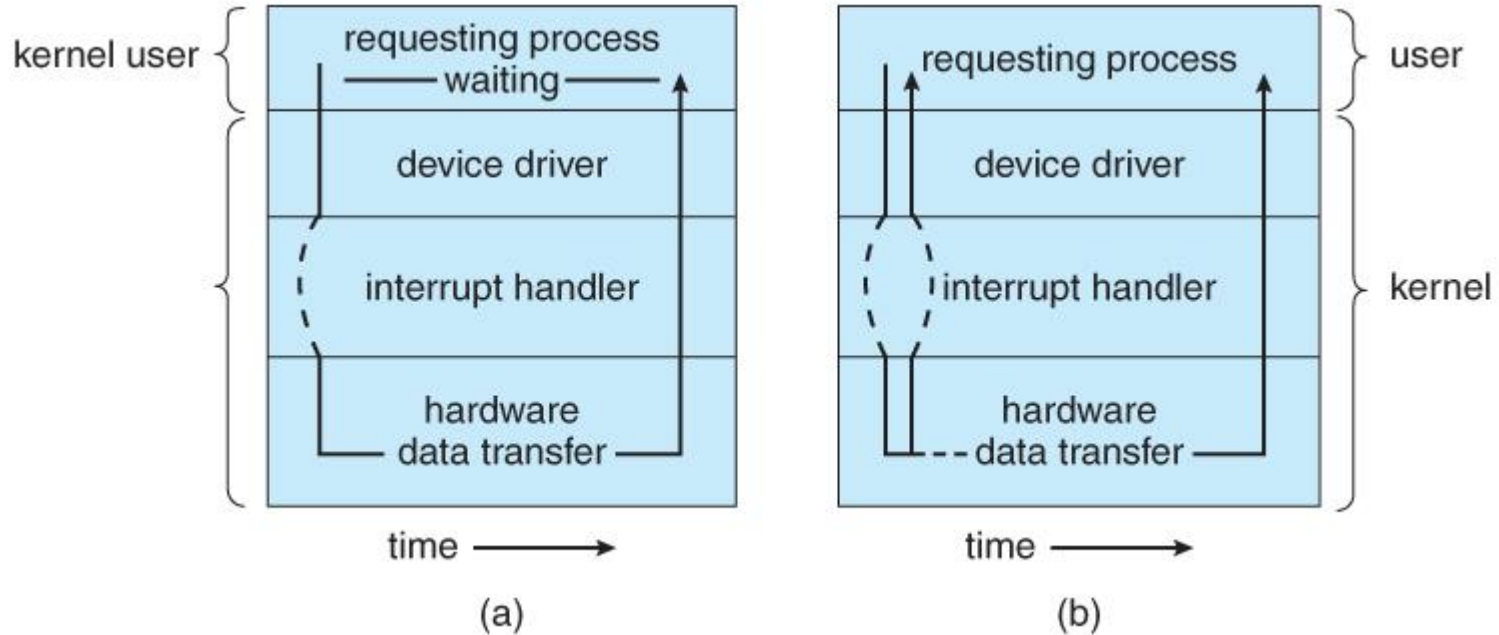
Block, Character and Network Devices

- *Block devices* are accessed a block at a time -> support include `read( )`, `write( )`, and `seek( )`.
 - Accessing blocks on a hard drive directly (without going through the filesystem structure) is called *raw I/O*
 - speed up certain operations by bypassing the buffering and locking -> application's responsibility
 - alternative -> *direct I/O* -> uses the normal filesystem access -> but disables buffering and locking operations.
- Memory-mapped file I/O can be layered on top of block-device drivers.
 - Rather than reading in the entire file, it is mapped to memory addresses, and then paged as needed
 - Access to the file through normal memory accesses -> executable program code
- *Character devices*-> one byte at a time ->`get( )` and `put( )`->advanced functionality, reading a line
- Network Device - inherently different from local disk access, most systems provide a separate interface for network devices.
 - *socket* interface ->like a cable or pipeline connecting two networked entities. Data can be put into the socket at one end, and read out sequentially at the other end->normally full-duplex, allowing for bi-directional data transfer.
 - The `select( )` system call allows servers to identify sockets which have data waiting, without having to poll all available sockets.

Blocking and Nonblocking I/O, Clock, Timer

- Three type of time services - current time, elapsed time since previous event, Set a timer to trigger event X at time T -> not standardized across all
- A *programmable interrupt timer, PIT* trigger operations, measure elapsed time, trigger an interrupt at a specific future time, or periodically.
 - Scheduler uses a PIT to trigger interrupts for ending time slices.
 - Disk system -> schedule periodic maintenance cleanup, such as flushing buffers to disk
 - Networks use PIT to abort or repeat operations that are taking too long to complete.
 - System clock is implemented by counting interrupts generated by the PIT / provide direct access to a high frequency hardware counter
- **Blocking I/O** a process is moved to the wait queue when an I/O request is made, and moved back to the ready queue when the request completes, allowing other processes to run in the meantime.
- **Non-blocking I/O** -> I/O request returns immediately, whether the requested I/O operation has (completely) occurred or not. This allows the process to check for available data without getting -> multi-threaded application, in which one thread makes blocking I/O calls (say to read a keyboard or mouse), while other threads continue to update the screen or perform other tasks.
- **Asynchronous I/O**, in which the I/O request returns immediately allowing the process to continue on with other tasks, and then the process is notified (via changing a process variable, or a software interrupt, or a callback function) when the I/O operation has completed and the data is available.
 - non-blocking I/O returns immediately with whatever results are available, but does not complete the operation and notify the process later.

Synchronous and Asynchronous I/O

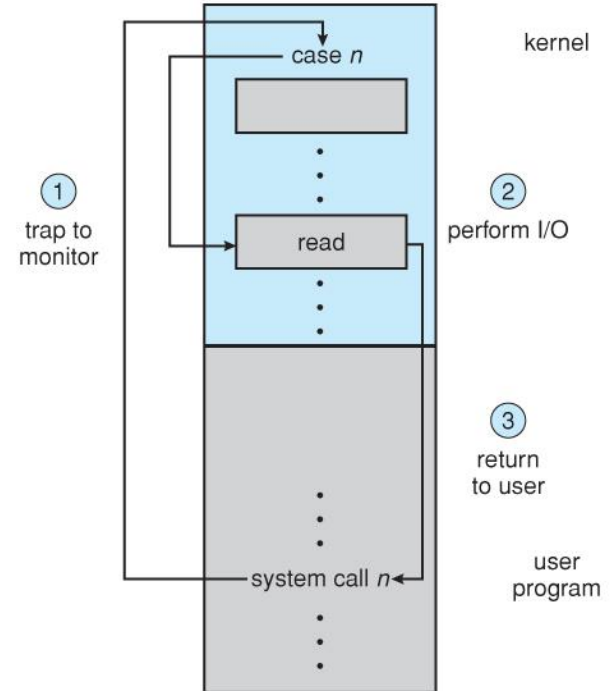


Kernel I/O Subsystem

- Scheduling I/O requests improve overall efficiency. Priorities -> request scheduling.
- Buffering -
 - Speed differences between two devices->Slow device write data into a buffer->buffer is full->entire buffer is sent to the fast device.
 - Slow device still has somewhere to write while this is going on, a second buffer is used, and the two buffers alternate as each becomes full. This is known as *double buffering*.
 - Double buffering is often used in (animated) graphics, so that one screen image can be generated in a buffer while the other (completed) buffer is displayed on the screen. This prevents the user from ever seeing any half-finished screen images.
 - Data transfer size differences. Buffers are used in networking systems to break messages up into smaller packets for transfer and re-assembly at the receiving side.
 - Copy Semantics -> disk write, the data is copied from the user's memory area into a kernel buffer->Application can change their copy of the data
- Caching -
 - Keeps a *copy* of data in a faster-access location than where the data is normally stored.
 - Buffer may hold the only copy of a given data item, whereas a cache is just a duplicate copy of some other data stored elsewhere.
 - Same storage space may be used for both purposes-> after a buffer is written to disk, then the copy in memory can be used as a cached copy

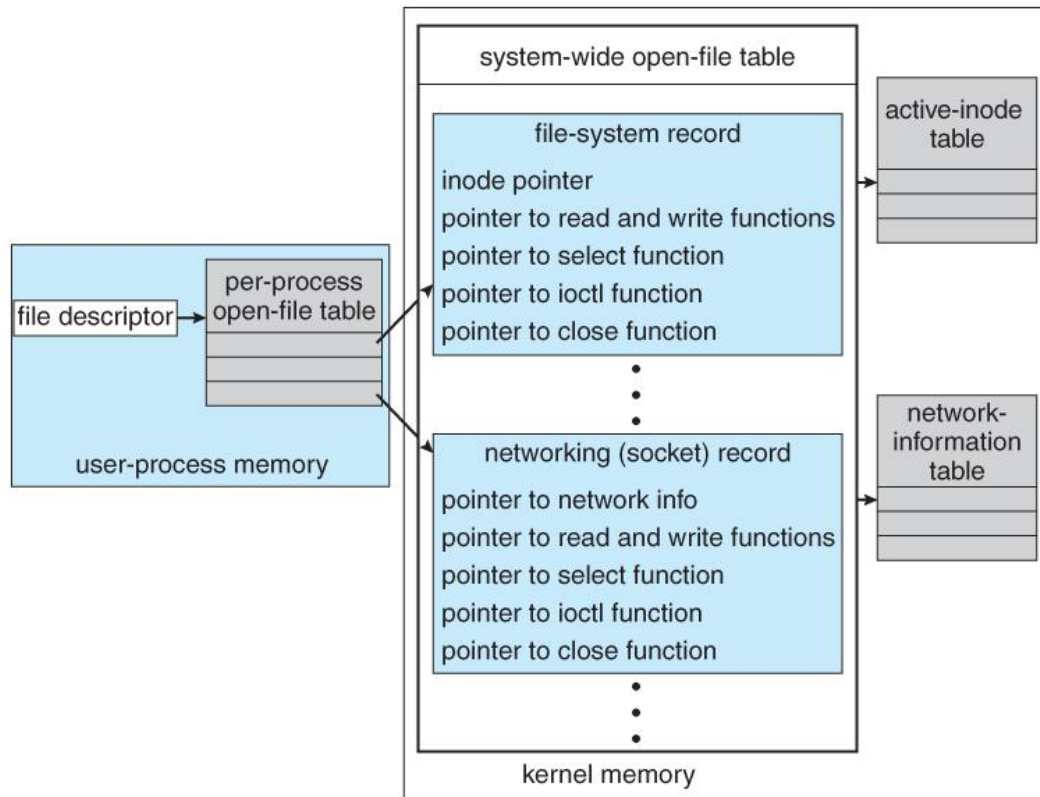
Kernel I/O Subsystem

- Spooling - *Simultaneous Peripheral Operations On-Line* buffers data for (peripheral) devices such as printers that cannot support interleaved data streams.
 - If multiple processes want to print at the same time->send their print data to files stored in the spool directory. When each file is closed, then the application sees that print job as complete, and the print scheduler sends each file to the appropriate printer one at a time.
 - Support->viewing the spool queues, removing jobs from the queues, moving jobs from one queue to another queue, and in some cases changing the priorities of jobs in the queues.
 - Request / get exclusive access to a particular device, and/or to wait until a device becomes available.
- Error Handling - I/O requests can fail for many reasons, either transient (buffers overflow) or permanent (disk crash).
 - Problem -> return an error bit (or more). set the global variable *errno* to well-defined values
 - On-board error log -> SCSI devices
- I/O Protection
 - The I/O system must protect against either accidental or deliberate erroneous I/O.
 - User applications are not allowed to perform I/O in user mode - All I/O requests are handled through system calls in kernel mode.



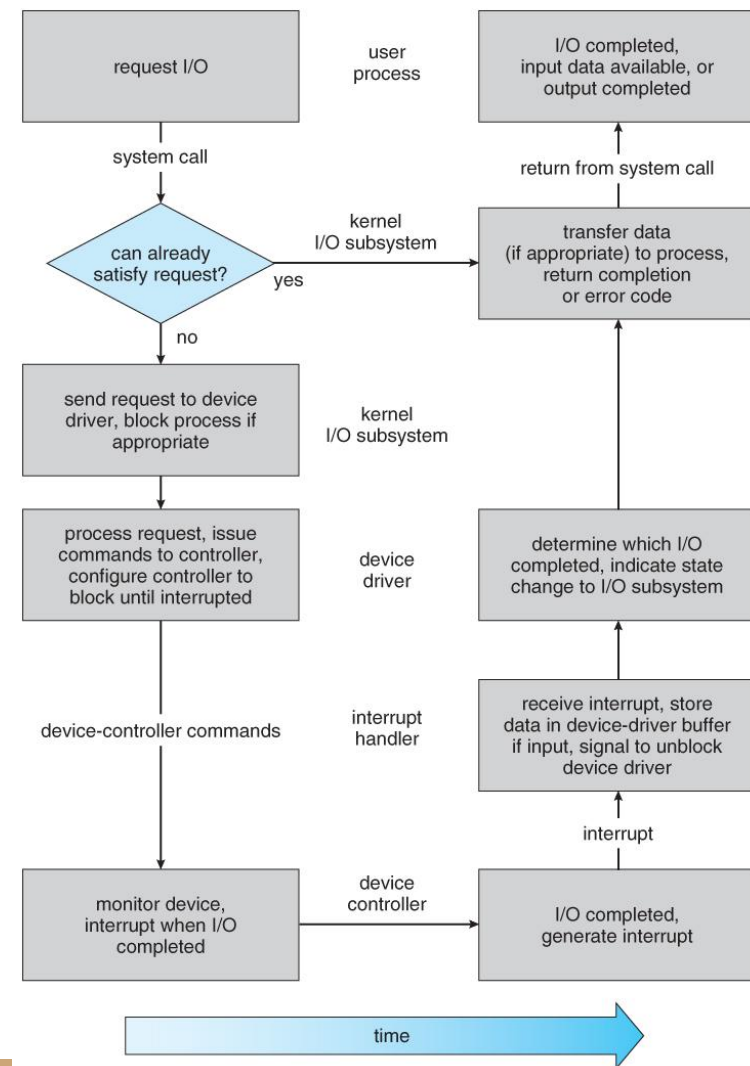
Kernel I/O Subsystem

- I/O Protection - Memory mapped areas and I/O ports must be protected -> access to these areas cannot be totally denied to user programs.
 - memory protection system restricts access so that only one process at a time can access particular parts of memory
- Kernel Data Structure - maintains a number of important data structures -> open file table.
 - object-oriented, and flexible to allow access to a wide variety of I/O devices through a common interface.
 - Windows NT carries the enhances by implementing I/O as a message-passing system from source -> various intermediaries -> device.



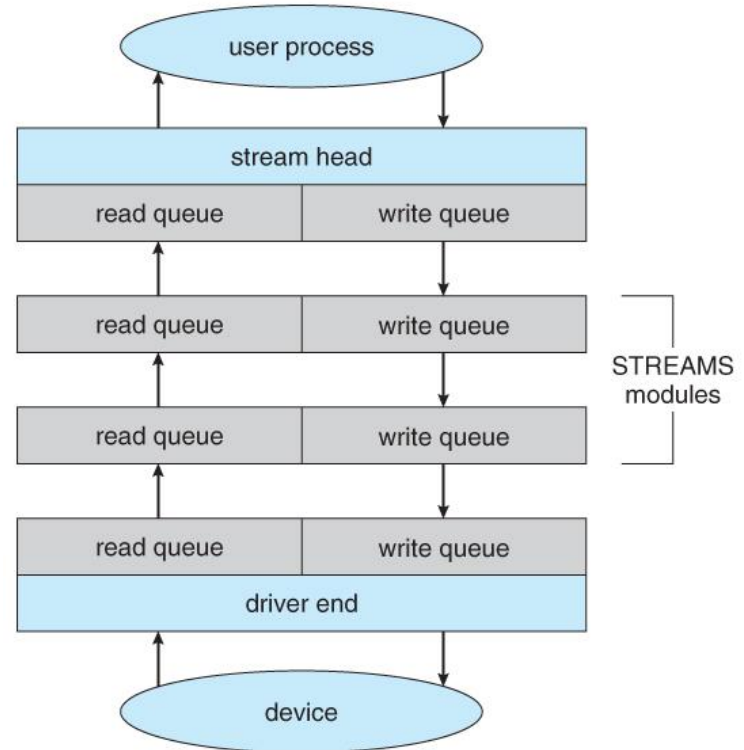
Transforming I/O Request to Hardware Operations

- Users request data using file names, which must ultimately be mapped to specific blocks of data from a specific device managed by a specific device driver.
 - DOS uses the colon separator to specify a particular device (e.g. C:, LPT:, etc.)
 - UNIX uses a *mount table* to map filename prefixes (e.g. /usr) to specific mounted devices.
 - multiple entries in the mount table match different prefixes of the filename the one that matches the longest prefix is chosen. (e.g. /usr/home instead of /usr where both exist in the mount table and both match the desired file.)
- UNIX -> *device files*, located in /dev -> represent, access physical devices directly.
 - Each device file has a major and minor number associated with it, stored and displayed where the file size would normally go.
 - The major number -> index into a table of device drivers, indicates the device driver handles the device. (E.g. the disk drive handler.)
 - The minor number -> parameter passed to the device driver, and indicates which specific device is to be accessed, out of the many which may be handled by a particular device driver. e.g. a particular disk drive or partition.
- A series of lookup tables and mappings makes the access of different devices flexible, and somewhat transparent to users



STREAMS (optional)

- The **streams** mechanism in UNIX provides a bi-directional pipeline between a user process and a device driver,
 - The user process interacts with the *stream head*.
 - The device driver interacts with the *device end*.
- Zero or more *stream modules* can be pushed onto the stream, using `ioctl()`. These modules may filter and/or modify the data as it passes through the stream.
- Each module has a *read queue* and a *write queue*.
- *Flow control* can be optionally supported -> each module will buffer data until the adjacent module is ready to receive it. Without flow control, data is passed along as soon as it is ready.
- User processes communicate with the stream head -> `read()` and `write()` (or `putmsg()` and `getmsg()` for message passing.)
- Streams I/O is asynchronous (non-blocking), except for the interface between the user process and the stream head.
- The device driver must respond to interrupts from its device - If the adjacent module is not prepared to accept data and the device driver's buffers are all full, then data is typically dropped.
- Streams are widely used in UNIX, and are the preferred approach for device drivers. For example, UNIX implements sockets using streams.



Performance (Optional)

- The I/O system is a major factor in overall system performance, and can place heavy loads on other major components of the system (interrupt handling, process switching, memory access, bus contention, and CPU load for device drivers just to name a few.)
- Interrupt handling can be relatively expensive (slow), which causes programmed I/O to be faster than interrupt-driven I/O when the time spent busy waiting is not excessive.
- Network traffic can also put a heavy load on the system. Consider for example the sequence of events that occur when a single character is typed in a telnet session, as shown in figure 13.15. (And the fact that a similar set of events must happen in reverse to echo back the character that was typed.) Sun uses in-kernel threads for the telnet daemon, increasing the supportable number of simultaneous telnet sessions from the hundreds to the thousands.
- Other systems use *front-end processors* to off-load some of the work of I/O processing from the CPU. For example a *terminal concentrator* can multiplex with hundreds of terminals on a single port on a large computer.
- Several principles can be employed to increase the overall efficiency of I/O processing:
 - Reduce the number of context switches.
 - Reduce the number of times data must be copied.
 - Reduce interrupt frequency, using large transfers, buffering, and polling where appropriate.
 - Increase concurrency using DMA.
 - Move processing primitives into hardware, allowing their operation to be concurrent with CPU and bus operations.
 - Balance CPU, memory, bus, and I/O operations, so a bottleneck in one does not idle all the others.
- The development of new I/O algorithms often follows a progression from application level code to on-board hardware implementation, as shown in Figure 13.16. Lower-level implementations are faster and more efficient, but higher-level ones are more flexible and easier to modify. Hardware-level functionality may also be harder for higher-level authorities (e.g. the kernel) to control.

